derived from “universal,” “uniform,” and “unique”: all of the world’s language needs are *universally* met, characters are represented *uniquely*, and the code units are *uniform* (meaning the code units are a fixed size; this is now definitely NOT the case as Unicode and its related transformation formats have evolved with time, as we shall see).

A key innovation of Unicode was the (eventual) abstraction of the code page from its machine representation. Rather than specify byte values for characters, each point in the Unicode character set simply has an ordinal number, which allows for flexibility in implementation across various machine architectures. An implementing system just needs to find a way to represent the integer codepoints; how that happens is not the concern of Unicode (anymore, but the original specification had more to say about it). Although not intended in its initial fixed-width design, it is this abstraction that allowed for variation in the *transformation format* and led to the efficiency and backwards compatibility of the variable-width *UTF-8*. It has also allowed Unicode to grow far beyond its initial scope of 65,536 characters. Today, Unicode (version 17.0) contains over one million possible codepoints covering all languages, with rules for their collation, and even languages that haven’t been spoken outside of an academic context for thousands of years (there are codepoints for Cuneiform, Ancient Egyptian, and Coptic scripts). Not all one million codepoints are assigned; there are large blocks of currently unused codepoints providing future-proofing.

PHP technically doesn’t have “native” Unicode support because, well, full support for the Unicode specification would be incredibly complex. Unicode doesn’t just offer millions of codepoints and the freedom to implement at the machine level; it also contains rules for combining characters (diacritical marks), bidirectional scripts (for when you’d like to mix left-to-right English with, say, right-to-left Hebrew… stay tuned for an example), truncating characters, and more. This complexity is necessary because human language is complex. However, PHP from version 7 onwards has offered expanded support for the use of Unicode in strings.

PHP 7.0 introduced the “Unicode codepoint escape syntax,” which means you can use the following meta-character sequence inside double-quoted string literals:

```
print "\u{5408} \u{6c23} \u{9053}" . PHP_EOL;
```

```
Output:
合 氣 道
```